

9. Übungsblatt zur Vorlesung Fortgeschrittene Algorithmen (WS 2025/26)

Aufgabe 1 – Randbeschriftungen

In der Vorlesung haben wir einen Algorithmus für 1-seitige Randbeschriftungen mit po-
Leadern und fixen Positionen der Beschriftungen kennengelernt, der für jede Eingabe
von n Punkten (in allgemeiner Lage) und n Rechtecken am rechten Rand des umschlie-
ßenden Rechtecks eine kreuzungsfreie Beschriftung minimaler Länge findet. Aber wie
schaut es aus, wenn die Rechtecke an mehr als einer Seite liegen?

- a) Beschreiben Sie einen effizienten Algorithmus, der für jede Eingabe von n Punkten
(in allgemeiner Lage) und n Rechtecken am *linken und rechten* Rand des umschlie-
ßenden Rechtecks eine kreuzungsfreie Beschriftung findet.

Ist die gefundene Lösung längenminimal?

4 Punkte

- b) Gibt es auch einen effizienten Algorithmus, der für jede Eingabe von n Punkten (in
allgemeiner Lage) und n Rechtecken am *oberen und rechten* Rand des umschließen-
den Rechtecks eine kreuzungsfreie Beschriftung findet?

2 Punkte

- c) Zeigen Sie, dass *jeder* Algorithmus, der eine kreuzungsfreie Beschriftung von n
Punkten mit po-Leadern und fixen Positionen der Beschriftungen berechnet, Lauf-
zeit $\Omega(n \log n)$ benötigt. Nehmen Sie dafür an, dass es einen Algorithmus gibt, der
das Problem in $o(n \log n)$ Zeit löst und zeigen Sie dann, wie man ihn benutzen kann,
um eine Menge von n Zahlen in $o(n \log n)$ Zeit zu sortieren.

4 Punkte

Aufgabe 2 – Suffixbäume und Suffixarrays

- a) Erstellen Sie einen Suffixbaum und ein Suffixarray für den String mississippi.

3 Punkte

- b) Sei T ein String über dem Alphabet Σ mit $n = |T|$ und sei M ein Muster mit $m = |M|$. Beschreiben Sie, wie man die Datenstrukturen aus der Vorlesung so anpassen
kann, dass sich die *Anzahl* der Vorkommen von M in T in $\mathcal{O}(m \log |\Sigma|)$ Zeit (also
unabhängig von k) finden lässt. Die Konstruktion der Datenstruktur sollte immer
noch in $\mathcal{O}(n)$ Zeit laufen.

3 Punkte

- c) Sei T ein String über dem Alphabet Σ mit $n = |T|$. Ein sich wiederholender Teilstring in T ist ein Infix $T[i, j]$, so dass es ein Infix $T[i', j'] = T[i, j]$ mit $i < j < i' < j'$ gibt. Beschreiben Sie einen Algorithmus, der einen längsten sich wiederholenden Teilstring in T in $\mathcal{O}(n)$ Zeit findet.

Hinweis: Verwenden Sie einen Suffixbaum oder ein Suffixarray von T . **4 Punkte**

Bitte geben Sie Ihre Lösungen bis **Mittwoch, 21. Januar 2026, 12:00 Uhr** einmal pro Gruppe über Moodle als pdf-Datei ab. Vermerken Sie dabei stets die Namen aller Bearbeitenden auf der Abgabe.

Grundsätzlich sind stets alle Ihrer Aussagen zu begründen und Ihr Pseudocode ist stets zu kommentieren.